最大流練習

26-E01 反平行流、淨流與殘餘

容量s→a=4、s→b=1、a→b=3、b→a=2、a→t=1、b→t=4。給非負arc流g_sa=3、g_sb=1、g_ab=3、g_ba=1、g_at=1、g_bt=3。核對守恆、換成淨流，求a→b與b→a殘餘；再構造一次增廣及等值cut，證明最後最優。不可對反對稱淨流另要求逐方向非負。

26-E02 流與割是不同方向的證書

容量s→a=5、s→b=3、a→b=2、a→t=3、b→t=4。構造最大流，逐點核對守恆與容量，給一個同值cut。若只知道source總出8與sink總入7，是否足以宣稱最大值7？說明缺少什麼資訊。

26-E03 偽多項式的實際選路族

取C為正整數，容量s→a=s→b=a→t=b→t=C、a→b=1。反覆交替選s→a→b→t與s→b→a→t。給第k對路徑後的五條淨流，證明恰需2C次增廣；若C=2^B，與輸入bit長的關係為何？同一圖以BFS最短路需要幾次？

26-E04 Matching、翻轉與 Hall 障礙

二分圖原邊L1R1、L1R2、L2R1、L3R2、L3R3。由matching{L1R1,L3R2}找交替增廣路，列翻轉後最大matching及一個同大小vertex cover。若刪L3R3，找出違反Hall的左側集合，證明不可能覆蓋全部左點。

26-E05 沒有殘餘 s-t 路仍可能未算完

三點s,u,t，容量s→u=2、u→t=1。用push-relabel初始化，列f、e、h及residual。此時是否有s-t殘餘路？為何不能直接由max-flow min-cut判定已得到最大flow？完整執行至終點，包含退回source的步驟。

26-E06 改變鄰居順序，不改最優值

使用正文五點容量網路，但改N(a)=[t,b,s]、N(b)=[t,c,a,s]、N(c)=[t,b,s]，L初值[a,b,c]。列每次push的量與excess、每個discharge後的list/current/height，給最後全部原邊流及最小cut。比較與正文的b→c、b→t、c→t是否必相同。

26-E07 Current 的安全性與成本

u高度保持不變，某uv在掃描時殘餘0而被略過。之後別點操作使uv殘餘恢復，為何仍不須立刻把current倒回uv？另證先前正殘餘卻不admissible的邊也不會自行變admissible。以此推導全部current advances的O(nm)，並說明若每次discharge開始都重設head，原計費證明哪一步失效。

最大流練習 （續）

26-E08 有理容量與精確證書

容量s→a=1/2、a→t=1/3、s→b=b→t=1/6。給最大流與最小cut；用共同分母6轉成整數後求值再還原。對一般binary有理輸入，解釋分母乘積可能數值指數大、bit長卻為多項式，並比較Ford–Fulkerson與Edmonds–Karp的終止輪數依賴。

26-E09 把路徑補成完整殘餘網路

所有原容量s→a、s→b、a→b、a→t、b→t皆為1。先走s→a→b→t，再走s→b→a→t，各增廣1。列每輪後所有正殘餘弧、每條原邊流量，並以可行流與等值割證明最優；為何最後仍有正殘餘a→b，卻不能再送流？另將容量依上述原邊順序改成(3,2,1,2,3)，依BFS（a先於b）列三輪的完整正殘餘弧、d(a,b,t)、路徑及瓶頸，直到終止。

26-E10 格點不能共用的逃生模型

5×5網格只開放{(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}，座標為(row,column)，其他格點封閉。兩人從(3,2)、(2,3)出發，可從開放邊界離開。分別求「邊不能共用」與「格點不能共用」的最多逃出人數；給具體路徑及對應的割。為一般k×k網格寫出拆點網路、頂點與邊數上界，證明整數最大流與格點不相交路徑的雙向對應，並處理k=1、起點已在邊界的情況。

26-E11 六點網路的反向撤流與割證書

原容量為sw:8、wx:9、xy:3、xt:4、sy:10、yz:7、zt:8。依序沿s-w-x-y-z-t、s-y-x-t、s-y-z-t、s-w-x-t增廣。每輪列增廣前路徑殘餘容量、δ、七條原邊流量與流值；說明第二輪的y→x何時出現。最後找殘餘可達集合、列全部跨割原邊，證明最優。若初始就選s-y-x-t，哪個合法性條件會失敗？

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study